

ErrPilot: Auditable Failure Provenance and Handoff for AI-Assisted Debugging

Yangchenxi Wu

Hungarian University of Sports Science

Budapest, Hungary

wuyang778@gmail.com

Abstract

ErrPilot is a CLI-first tool for turning explicitly wrapped failing commands into auditable failure provenance for AI-assisted debugging workflows. Rather than acting as a repair agent, ErrPilot standardizes the intake layer before a human or downstream coding tool investigates a failure: what command failed, what raw evidence was captured, what source context was selected, what local routing decision was made, and what verification contract accompanies a handoff. It captures command metadata, preserves raw stdout/stderr logs while exposing compact structured excerpts, parses Python traceback and pytest failure signals, extracts bounded source contexts, runs deterministic local triage, and generates reviewable handoff prompts. The artifact is local-first and reproducible: it requires no external service calls, does not silently monitor terminal activity, and does not execute downstream repair tools. We evaluate ErrPilot with 7 open-source, repository-local executable failure cases covering pytest assertions, fixture/setup failures, imports, multi-test failures, missing configuration, type errors, and standalone tracebacks. A deterministic handoff-quality probe on all 7 cases shows that raw failure prompts contain only 2/7 required handoff fields on average, while ErrPilot handoff artifacts contain 7/7. We also report one optional cross-project vignette on the public SkiLoad-Lab Python CLI. Artifact: <https://doi.org/10.5281/zenodo.20053341>. Video: <https://youtu.be/-cGRD9A6AXo>.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *General programming languages*.

Keywords

failure intake, debugging, software artifacts, triage, AI-assisted software engineering

ACM Reference Format:

Yangchenxi Wu. 2026. ErrPilot: Auditable Failure Provenance and Handoff for AI-Assisted Debugging. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern AI-assisted debugging workflows often begin from an unstructured failure: a test run, traceback, build log, or CI excerpt is pasted into a human review or coding-agent session [6]. This intake step is easy to under-specify. The prompt may omit command

metadata, lose raw evidence, include excessive logs, or fail to record the local source context and verification contract needed to audit a suggested repair.

ErrPilot addresses this intake problem. It is a CLI-first failure provenance and handoff tool that turns explicitly wrapped failing commands into structured, auditable records. ErrPilot is not a repair agent. It does not modify source code, apply patches, execute downstream coding tools, or silently monitor terminal activity. Instead, it captures the failure, extracts structured local evidence, runs deterministic local triage, and emits reviewable handoff prompts for humans or downstream tools.

ErrPilot complements existing failure surfaces rather than replacing them. Terminal output and CI logs preserve evidence, but they do not provide a stable, structured intake object. Issue templates encourage structure, but rely on manual completion. Coding agents and automated repair systems can consume failure evidence [6, 7], but the evidence is often pasted ad hoc. ErrPilot addresses the intermediate layer: it constructs a repeatable provenance object before repair or investigation begins, so later actions can be traced back to captured evidence and an explicit handoff record.

This demonstration paper makes four contributions:

- a structured failure provenance schema separating raw evidence, parsed diagnostics, bounded source contexts, triage records, and handoff artifacts;
- a CLI workflow for explicit run capture, bundle construction, deterministic local triage, and reviewable handoff prompt generation;
- a bounded source-context extraction policy for Python traceback and pytest failures without copying complete source files into the bundle; and
- a reproducible artifact-readiness evaluation with 7 executable local cases, a deterministic handoff-quality probe quantifying provenance completeness over raw prompts, one optional cross-project vignette, documented design probes, CI checks, and optional Docker validation.

2 Tool Overview

ErrPilot is used as an explicit wrapper around a command selected by the user:

```
errpilot run -- pytest \
  examples/python_assertion_failure
errpilot bundle latest
errpilot triage latest --local
errpilot route latest --target codex
```

The first command records stdout, stderr, a combined log, command text, exit status, and timing metadata under a run directory.

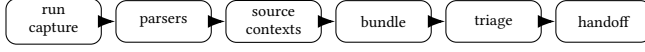


Figure 1: ErrPilot failure intake pipeline from explicit command capture to structured bundle generation, deterministic triage, and reviewable handoff prompt creation.

Table 1: Failure bundle schema groups. Raw evidence remains on disk while the JSON bundle exposes compact, auditable fields for tools and reviewers.

Group	Fields	Purpose
Run	command, cwd, exit_code, run_id	failed command context
Logs	stdout/stderr paths, compact excerpts	raw evidence plus compact review view
Diagnostics	python_traceback, pytest, failing_tests	parsed failure signals
Context State	source_contexts, git	bounded code windows, branch, commit, dirty status
Handoff	triage, handoff_artifacts	routing and prompt outputs

Subsequent commands construct a human-readable and machine-readable bundle, expose repository state in the bundle, attach local triage, and write a target-specific prompt artifact. ErrPilot executes only the user-supplied command explicitly passed to `errpilot run -;` it does not sandbox that command.

3 Failure Bundle Schema and Workflow

ErrPilot stores raw evidence as files and exposes structured diagnosis fields in `error_bundle.json`. The stable top-level fields include run metadata, the wrapped command, log paths and compact excerpts, git state, parsed Python traceback information, pytest failing tests, bounded source contexts, risk flags, local triage, and handoff artifact records. A companion `error_bundle.md` provides a readable review surface.

The bundle is intentionally organized for auditability. Raw logs remain available as separate files, while prompt artifacts use structured fields and bounded excerpts. Very small logs may fit entirely inside an excerpt; the design goal is bounded evidence, not a guarantee that no complete small log can appear. Sensitive paths such as `.env`, private keys, token files, and credential files are excluded from source-context collection by default.

Design policies. ErrPilot applies deterministic intake policies rather than learned ranking. For pytest output, it records failing node identifiers, file paths, test names, and error classes when those signals are present. For Python tracebacks, it selects repository-local frames and ignores external library frames for source-context collection. Source windows are bounded around the failing line and carry a role label such as `failing_test` or `traceback`; complete files are not copied into handoff prompts. Local triage applies ordered rules over parsed signals such as missing imports, missing files, fixture/setup failures, multi-test failures, and local assertion or type errors. Handoff generation then renders the structured bundle

Table 2: Positioning against common failure-handling workflows.

Surface	Strength	Gap addressed by ErrPilot
Terminal / CI logs	preserve raw output	no reusable structured bundle
Issue templates	encourage structure	manually completed evidence
Agent prompts	immediate handoff	weak audit trail
Repair agents	can investigate or patch	execute after intake; not a record format
ErrPilot	structured local intake	stops before repair, stores auditable artifacts

into a target-specific artifact that includes a verification command and constraints, but does not invoke the target.

Local triage is deterministic and rule-based. It records severity, confidence, recommended route, reason, and a human-approval requirement. Route generation then writes prompt artifacts for targets such as Codex, Aider, Gemini CLI, or manual review. These prompt files are artifacts for inspection and handoff; ErrPilot does not execute downstream repair tools.

4 Evaluation and Artifact Readiness

We provide a reproducible artifact-readiness evaluation augmented with a deterministic handoff-quality probe that quantifies the provenance advantage of ErrPilot handoff artifacts over raw failure prompts. The evaluation asks whether each executable failure can be captured, normalized, triaged, routed, and reproduced without relying on external services or downstream repair tools.

For the handoff-quality probe, we run the same checklist over all 7 executable failures. The probe compares a raw-log prompt against the generated Codex handoff artifact and checks whether the input explicitly contains seven fields needed for a controlled debugging handoff: expected error, relevant file, verification command, source context, triage route, human-approval gate, and safety constraints. This probe does not ask an LLM or human to repair the bug; it measures whether the handoff artifact carries the evidence and guardrails that a downstream workflow would otherwise need to reconstruct.

This demonstrates that ErrPilot systematically closes a provenance gap that raw failure logs leave for controlled downstream debugging handoff.

The 7 executable cases are open-source examples maintained inside the artifact repository and are the primary reproducible evidence. They cover pytest assertions, fixture/setup failures, import failures, multi-test failures, missing configuration, type errors, and standalone Python tracebacks. All 7 produce raw run artifacts, structured bundles, bounded source contexts, local triage records, and reviewable handoff artifacts.

We additionally ran an optional cross-project vignette against the public `SkiLoadLab` Python package [2]. The command invoked its documented `skiloadlab.cli.combine` entry point with a deterministic missing CSV input under `/tmp`. ErrPilot captured the file-missing traceback, selected repository-local contexts in the CLI

Table 3: Artifact-readiness criteria exercised by the 7 executable open-source, repository-local cases.

Criterion	Evidence checked	Coverage
Capture	logs, command, metadata, exit code	7/7 cases
Parsing	pytest failures or Python traceback fields	7/7 cases
Context	bounded repository-local source windows	7/7 cases
Triage	severity, route, confidence, approval flag	7/7 cases
Handoff	prompt artifact plus verification command	7/7 cases
Reproduction	tests, lint, evaluation, demo path	check script

Table 4: Deterministic handoff-quality probe on all 7 executable cases. Scores count explicit fields present in the input artifact, not repair success.

Input artifact	Mean score	Missing from raw log
Raw failure prompt	2.0/7	verification contract; bounded source context; triage and safety constraints
ErrPilot handoff	7.0/7	no checked provenance fields missing

wrapper and core model, produced local triage with severity 4 and a human-approval gate, and generated a Codex handoff artifact. This is a vignette, not a repair benchmark: it demonstrates that the intake workflow transfers to a separate open-source Python CLI without requiring external services.

The remaining documented-only external entries are design probes from SkiLoadLab and slsa-verifier contexts. They are included to motivate CI, shell, and supply-chain failure categories for future work. They are not executed by default, avoid external repository paths, archived CI state, network calls, and mutation, and are excluded from all default executable artifact-readiness claims.

Artifact readiness is checked in three paths: a clean-clone virtual environment path that runs tests, lint, evaluation, and a minimal CLI demo; a one-command artifact-check script covering the same steps; and an optional Docker path that builds a local image and runs the artifact check without publishing images or requiring Docker Hub credentials. The GitHub Actions workflow separates the Python validation job from the Docker validation job [5]. The Docker path follows common containerized artifact practice [4], but it is optional; the clean clone plus virtual environment path remains the primary reproduction route.

The artifact is archived on Zenodo and is intended to satisfy the spirit of ACM artifact availability guidance: reviewers can retrieve a versioned object with a DOI and exercise the included scripts without external services [8]. The version-specific snapshot is separate from the live GitHub repository so that paper claims can be tied to a stable release.

At the time of writing, the public repository includes a passing Python CI job, a passing Docker validation job, a Zenodo archive, and a one-command artifact check script.

The executable cases are intentionally small and deterministic, which improves artifact reproducibility but limits ecological validity. They are open-source cases in the artifact repository rather than closed synthetic fixtures, but they do not establish downstream repair success or field performance. The SkiLoadLab vignette is a single cross-project sanity check, not evidence of general field performance. The documented external probes motivate broader categories only.

5 Limitations

ErrPilot does not repair bugs, rank patches, or measure downstream agent success. It standardizes failure intake and handoff before repair, rather than competing with repair agents. The evaluation is an artifact-readiness study, not a broad empirical benchmark or downstream-agent comparison. The external SkiLoadLab vignette is optional and single-case; slsa-verifier entries are documented-only design probes and are not reproduced by the default evaluation. Local triage is deterministic and intentionally simple. ErrPilot executes user-supplied commands explicitly passed to `errpilot run`; it does not sandbox those commands. Future work should measure downstream agent diagnosis and repair outcomes with and without ErrPilot bundles.

6 Data Availability Statement

ErrPilot, its documentation, executable examples, evaluation case metadata, generated evaluation results, artifact reproduction scripts, CI workflow, and optional Docker reproduction path are available in the public repository at <https://github.com/YangchenxiWu/errpilot> and archived on Zenodo at <https://doi.org/10.5281/zenodo.20053341>. The default artifact evaluation uses only in-repository examples and does not require external repositories, external APIs, secrets, or downstream repair tools. The SkiLoadLab vignette is provided as an optional cross-project check, while other external cases remain documented-only metadata for context and are not executed by default.

References

- [1] Yangchenxi Wu. ErrPilot artifact repository. Version-specific archival artifact snapshot: <https://doi.org/10.5281/zenodo.20053341>, 2026.
- [2] Yangchenxi Wu. SkiLoadLab: reproducible training load modeling for downhill skiing. <https://github.com/YangchenxiWu/SkiLoadLab>, 2026.
- [3] pytest developers. pytest documentation. <https://docs.pytest.org/>.
- [4] Docker Inc. Docker documentation. <https://docs.docker.com/>.
- [5] GitHub. GitHub Actions documentation. <https://docs.github.com/actions>.
- [6] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems*, 37, 2024. <https://doi.org/10.52202/079017-1601>.
- [7] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. <https://doi.org/10.1109/TSE.2011.104>.
- [8] ACM Publications Board. Artifact review and badging policy. <https://www.acm.org/publications/policies/artifact-review-and-badging-current>.